

Sistema de Gestão Escolar



Felipe Gonçalves Vidal, 202502208, felipevidal@discente.ufg.br
Rafael José de Souza Marques, 202502252, souza_rafael@discente.ufg.br
Robert Francisco Taveira, 202502255, robert_taveira@discente.ufg.br
Paulo de Tarso Rezende Lôbo, 202502247, rezende.lobo@discente.ufg.br



Visão Geral do Projeto

- **O que é:** Uma micro aplicação de gestão acadêmica executada inteiramente via Linha de Comandos (CLI).
- **O Objetivo:** Gerir o fluxo de informações de uma escola (Alunos, Professores, Turmas, Notas) com máxima eficiência.
- **A Abordagem:** Foco absoluto no dado. Sem interfaces gráficas pesadas; a operação é rápida, limpa e vai direto ao ponto.
- **Como funciona:** Tudo gira em torno de uma base de dados relacional que dita as regras do sistema.

Paradigma Orientado a Dados

- **Dado Como Verdade:** Ao invés de programar regras lógicas rígidas e sequenciais no código fonte, a própria modelagem do banco (esquema) atua como o motor do sistema.
- **Consistência Pura:** Em um sistema escolar, a integridade é vital. Este paradigma impede dados órfãos (ex: aluno vinculado a uma turma que foi deletada) na raiz.
- **Acoplamento Inteligente:** A aplicação (Python) serve apenas como um maestro ou interface. As regras de negócio pesadas e as validações moram no banco de dados.

Tecnologias Usadas

- **Python (Orquestrador):** A linguagem escolhida atua como a interface de comunicação. Ela captura a entrada do usuário na CLI, formata as requisições e processa as respostas. É a camada de apresentação e roteamento, mantendo o código "main" extremamente limpo e livre de lógicas complexas de validação estrutural.
- **SQLite3 (Motor):** O SGBD não é apenas um local de armazenamento; ele é o cérebro da aplicação. O SQLite foi escolhido por ser nativo, não exigir configuração de servidores externos e, crucialmente, por suportar as propriedades ACID. É o banco que "diz" se uma operação é válida ou não.

Princípios ACID na Prática

- **Atomicidade:** Transações de lançamento de nota ocorrem de forma integral; se a luz cair, a nota não fica pela metade.
- **Consistência:** Uso rigoroso do ON DELETE CASCADE. Se um professor é deletado, a sua vinculação com as turmas é automaticamente destruída pelo banco, não pelo Python.
- **Isolamento:** Operações simultâneas não interferem umas nas outras. O banco garante que a geração de um relatório de turma, por exemplo, não seja afetada ou bloqueada por um lançamento de notas acontecendo ao mesmo tempo.
- **Durabilidade:** O uso do modo de escrita WAL (Write-Ahead Logging) assegura que as notas nunca se percam após confirmadas.

Organização do Programa

- **Separação de Responsabilidades:** Para abraçar o paradigma de dados, isolamos a lógica CRUD. A interface (CLI) nunca toca diretamente no SQL.
- **Repositórios Isolados:** Criamos módulos dedicados (`aluno_repo.py`, `professor_repo.py`). O "main" pede ao repositório para salvar um aluno, e o repositório conversa com o banco.
- **Estrutura DDL Independente:** O arquivo `schema.sql` guarda o "plano arquitetônico" das tabelas isoladamente.

Lógica Comandada pelo Banco

- **Validação de Capacidade:** O limite de 30 alunos por turma não é apenas um "if" solto. O Python delega ao SQLite (SELECT COUNT(*)) a responsabilidade de contar e aprovar a inserção.
- **Cálculo de Situação:** O sistema define o aluno como "Aprovado" ou "Reprovado" com base em uma regra de negócio simples, que no futuro pode ser convertida diretamente em uma view ou trigger no SGBD.

$$\text{Situação} = \begin{cases} \text{Aprovado,} & \text{se } \frac{P_1 + P_2}{2} \geq 5.0 \\ \text{Reprovado,} & \text{caso contrário} \end{cases}$$

Melhorias Futuras

- **Triggers (Gatilhos):** Delegar o cálculo das notas P1 e P2 para o próprio SQLite através de Triggers, automatizando a mudança de "Cursando" para "Aprovado" no instante do UPDATE, sem o Python precisar processar.
- **Views para Relatórios:** Ao invés de fazer joins complexos no código Python, criar Views no banco de dados para gerar os relatórios de "Média Geral" e "Boletins", tratando as tabelas consolidadas como se fossem tabelas **simples**.
- **Stored Procedures:** Implementar procedures para matricular alunos em lote, movendo lógicas de repetição do lado do cliente (CLI) para dentro do servidor de banco de dados, aumentando a performance.

Fontes

Repositório: <https://github.com/plusfrapp/gerenciador-de-alunos>

ACID: <https://pt.wikipedia.org/wiki/ACID>

SQLite: <https://pt.wikipedia.org/wiki/SQLite>

Python: <https://pt.wikipedia.org/wiki/Python>

Obrigado!